ARTEFACTOS DEL MÓDULO

Punto de Control 3 (PC3)

Fase de Elaboración: arquitectura, contratos y diseño del módulo

Pacíficos Online — IF6100

INFORMACIÓN DEL SUBGRUPO

Subgrupo y módulo	SG5 — M5 — Carrito y Gestión de Pedidos
Integrantes	Manuel Salvador Ortiz Mejicano, C4H071 (Coordinador y Representante ante el Comité) Marcos José Ortega Fernández, C35750 (Arquitectura y Modelado) Edgar Emmanuel Duarte Alemán, C4E773 (Base de Datos e Integración) Genesis Lucia Gutierrez Espinoza, C4F794 (Diseño de Patrones) Ricardo Josué Castillo Acevedo, C21785 (Interacción y Prototipado UI)
Representante ante el Comité	Manuel Salvador Ortiz Mejicano
Fecha de entrega	Jueves 22 de octubre de 2026
Versión del documento	1.0 — Oficial

1. Dependencias del módulo

Antes de diseñar, el subgrupo deja claro qué necesita de otros módulos, qué les ofrece y qué toma de los artefactos del Comité. Esta sección alimenta el contrato (sección 2), los diagramas de interacción (sección 5) y los dobles de prueba (sección 9).

1.1 Dependencias con los artefactos del Comité (PC3)

Cada artefacto del subgrupo usa algo que define el Comité. Si el Comité aún no lo ha publicado, el subgrupo anota el supuesto que usa y lo ajusta cuando esté disponible.

Artefacto del subgrupo	Depende de (Comité PC3)	Qué toma de ahí	Estado
2. Contrato OpenAPI	4.1 Reglas de publicación de contratos; carpeta contratos/ del repositorio; 5.5 Manejo de errores	Formato, esquema de versionado, ubicación del archivo, estructura estándar de error (codigo, mensaje, detalles)	*Disponible*
3 y 4. Realización y clases de diseño	1.5 Vista lógica (capas); 5. Núcleo compartido	Capas internas que siguen todos los módulos; clases del núcleo para identidad, roles y aislamiento	*Pendiente*
5. Diagramas de interacción	4.2 y 4.3 Contratos de otros módulos; 1.6 Vista de procesos	Operaciones y eventos disponibles; qué interacciones son síncronas o por eventos	*Pendiente*
6. Modelo de datos y migraciones	6. Modelo de datos consolidado; 1.8 Convenciones de implementación; 1.9 Decisiones de datos	Tablas reservadas, claves compartidas, convención de nombres, uso o no de llaves foráneas entre módulos	*Disponible*
7. Patrones de diseño	3. ADR	Decisiones transversales que los patrones del módulo no deben contradecir	*Disponible*
8. Prototipo de interfaz	ADR del frontend; carpeta frontend/ y componentes compartidos	React + Tailwind CSS, organización por módulo, componentes comunes	*Disponible*
9. Dobles de prueba	4. Contratos publicados de otros módulos	Versión del contrato que el doble debe imitar	*Disponible*

1.2. Lo que el subgrupo entrega al Comité

El Comité necesita estos insumos para cerrar sus artefactos. El representante ante el Comité es el responsable de entregarlos.

Insumo	Destino en el Comité	Fecha	Entregado
Lista de tablas corregida del módulo	6. Modelo de datos consolidado	Jue 1 oct	*Si*
Borrador del contrato (operaciones y eventos)	4. Contratos publicados	Sáb 3 oct	*Si*
Contrato OpenAPI en la carpeta contratos/	4. Contratos publicados	*Sáb 3 oct*	*Si*
Módulo desplegado en el ambiente (implementación mínima)	7. Ambiente de integración	*Mie 21 oct*	*Si*

2. Contrato formal del módulo (OpenAPI)

Descripción formal de las operaciones y eventos que el módulo ofrece, versionada y publicada en el repositorio.

2.1 Datos del contrato

Versión	v1.1.0
Archivo en el repositorio	contracts/m5-pedidos/v1/openapi.yaml
Versión de OpenAPI	3.0.3
Estado	Publicado
Consumidores	Frontend del portal (M1), M2 (Comercios), M6 (Pagos y facturación), M7 (ERP Contable)

2.2 Operaciones

Método	Ruta	Descripción	Entrada	Respuesta exitosa	Errores
GET	/api/m5/v1/carrito	Devuelve el carrito activo del comprador y sus ítems.	Token de cliente	200 OK: Carrito	401 Unauthorized
POST	/api/m5/v1/carrito/items	Agrega una variación e ítem a items_carrito.	JSON: { id_variacion, cantidad, id_reserva }	201 Created: Carrito actualizado	401, 422 Unprocessable
DELETE	/api/m5/v1/carrito/items/{id_item_carrito}	Quita un registro de items_carrito.	Parámetro URL: id_item_carrito	200 OK: Carrito actualizado	401, 404 Not Found
POST	/api/m5/v1/pedidos/checkout	Genera pedidos_maestros y sus pedidos_subpedidos.	JSON: { cupon_descuento, tarjeta_regalo, nota }	201 Created: PedidoMaestro	400, 401, 409 Conflict (reserva fallida)
GET	/api/m5/v1/pedidos/{id_pedido}	Obtiene el detalle de una orden y sus subpedidos.	Parámetro URL: id_pedido	200 OK: PedidoMaestro	403 Forbidden, 404 Not Found
PATCH	/api/m5/v1/subpedidos/{id_subpedido}/cancelar	Cancela un subpedido y registra en cancelaciones_pedido.	Parámetro URL: id_subpedido. JSON: { motivo }	200 OK: Subpedido con estado cancelado	400 Bad Request, 404 Not Found

2.3 Eventos publicados

Evento	Versión	Cuándo se publica	Carga útil
PedidoConfirmado	v1.0.0	Al completar el checkout con éxito (estado confirmado).	pedido_id, monto_total, subpedidos: [{ subpedido_id, comercio_id, subtotal, costo_envio, impuesto, items: [{ id_variacion, cantidad, id_reserva }] }]
PedidoPagado	v1.0.0	Cuando M6 confirma el pago (estado pasa a pagado).	pedido_id, monto_total, referencia_pago, fecha_pago, subpedidos: [{ subpedido_id, comercio_id, subtotal }]
SubpedidoDespachado	v1.0.0	Cuando una tienda despacha su porción del pedido.	subpedido_id, pedido_id, comercio_id, fecha_despacho, guia_seguimiento
SubpedidoCancelado	v1.0.0	Si se anula un subpedido antes de entrega.	subpedido_id, pedido_id, comercio_id, motivo, requiere_reembolso, fecha_cancelacion

2.4 Esquemas expuestos

ItemCarrito (basado en items_carrito):

- id_item_carrito (int): Identificador de la línea del carrito.
- id_variacion (int): Variación de catálogo seleccionada.
- cantidad (int): Unidades seleccionadas.
- id_reserva (uuid, opcional): Identificador temporal de reserva asignado por M4.

Carrito (basado en pedidos_carritos + items_carrito):

- id_carrito (int): Identificador del contenedor activo del cliente.
- id_cliente (int): Cliente propietario del carrito.
- items (array de ItemCarrito): Colección de variaciones añadidas.

LineaPedido (basado en pedidos_lineas):

- id_linea (int): Identificador de la línea comprada.
- id_variacion (int): Producto/variación adquirida.
- cantidad (int): Cantidad comprada.
- precio_unitario (decimal): Precio unitario fijado al confirmar compra.
- tarifa_impuesto (decimal): Porcentaje de IVA aplicable.

Subpedido (basado en pedidos_subpedidos):

- id_subpedido (int): Identificador de la porción correspondiente a un comercio.
- id_comercio (int): Comercio responsable del despacho.
- subtotal (decimal): Suma de productos antes de impuestos y flete.
- costo_envio (decimal): Tarifa de transporte asociada a este comercio.
- impuesto (decimal): Monto total de IVA correspondiente.
- estado_subpedido (string): Condición particular (ej. pendiente, despachado, entregado).
- lineas (array de LineaPedido): Detalle de artículos que despachará esta tienda.

PedidoMaestro (basado en pedidos_maestros):

- id_pedido (int): Identificador global de la orden.
- fecha_creacion (date-time): Momento de confirmación de la transacción.
- monto_total (decimal): Suma unificada total a cobrar al cliente.
- estado_pedido (string): Condición global (confirmado, pagado, cancelado).
- cupon_descuento (string, opcional): Código de descuento global aplicado.
- tarjeta_regalo (string, opcional): Instrumento de pago complementario.
- subpedidos (array de Subpedido): Desglose de subpedidos asociados.

CancelacionPedido (basado en cancelaciones_pedido):

- id_cancelacion (int): Identificador del registro formal de anulación.
- id_pedido (int): Pedido maestro afectado.
- motivo (string): Razón documentada de la interrupción.
- requiere_reembolso (bool): Flag operativo para notificar devolución a M6.
- fecha_cancelacion (date-time): Momento en que se ejecuta la cancelación.

2.5 Historial de versiones del contrato

Versión	Fecha	Cambio	¿Rompe consumidores?
v1.0.0	03/10/2026	Versión inicial de contratos síncronos y asíncronos para PC3	No
V.1.1.0	16/10/2026	Correcciones para ajustarse a otros módulos	No

3. Realización de casos de uso

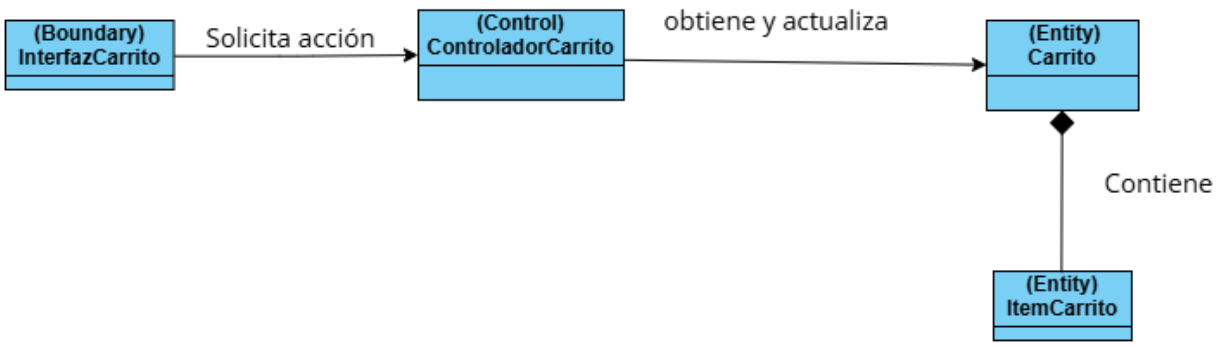
Muestra cómo las clases del módulo cumplen un caso de uso, separando clases límite, control y entidad.

3.1 Realización del caso de uso

Caso de uso (PC2)	CU-001 Gestionar carrito de compras
Actor principal	Cliente
Módulos externos involucrados	M4 Inventario para verificar existencias y M3 Catálogo para validar detalles del producto

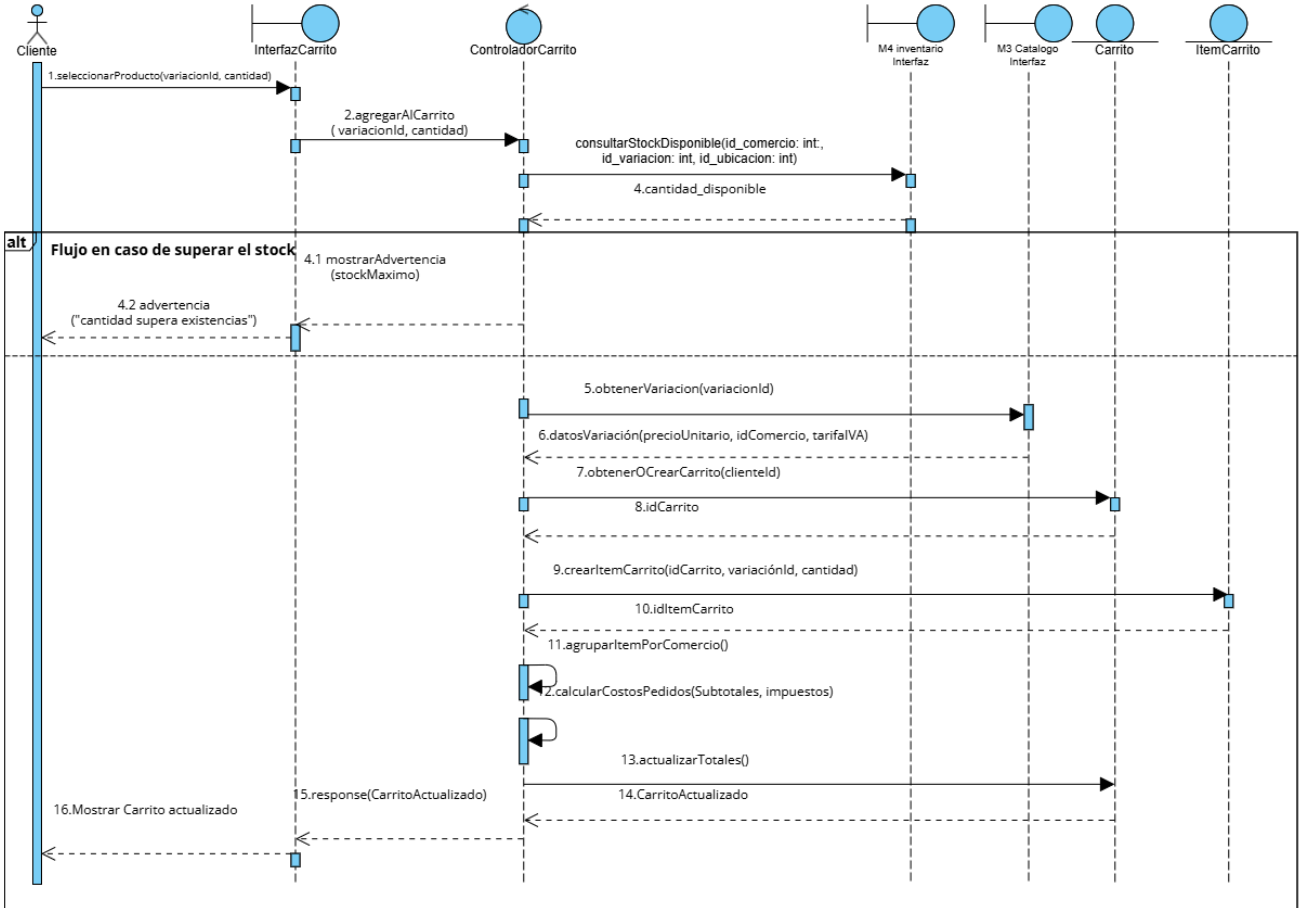
Estereotipo	Clase	Responsabilidad en este caso de uso
Límite	InterfazCarrito	Recibir las peticiones HTTP que realice un cliente para realizar las diferentes acciones como ver, agregar, actualizar, o eliminar ítems del carrito.
Control	ControladorCarrito	Coordinar la lógica de negocios para manipular el carrito lo que conlleva interactuar con el catálogo pudiendo asignar el estado actual.
Entidad	Carrito	Representar el carrito de compras del usuario en la base de datos.
Entidad	ItemCarrito	Representar cada uno de los artículos individuales que conforman el carrito añadido por el cliente.

Figura 1. *Diagrama de clases de CU-001 Gestionar carrito de compras*



Nota: Elaboración propia en herramienta Visual Paradigm

Figura 2. *Diagrama de secuencia de CU-001 Gestionar carrito de compras*



Nota: Elaboración propia en herramienta Visual Paradigm

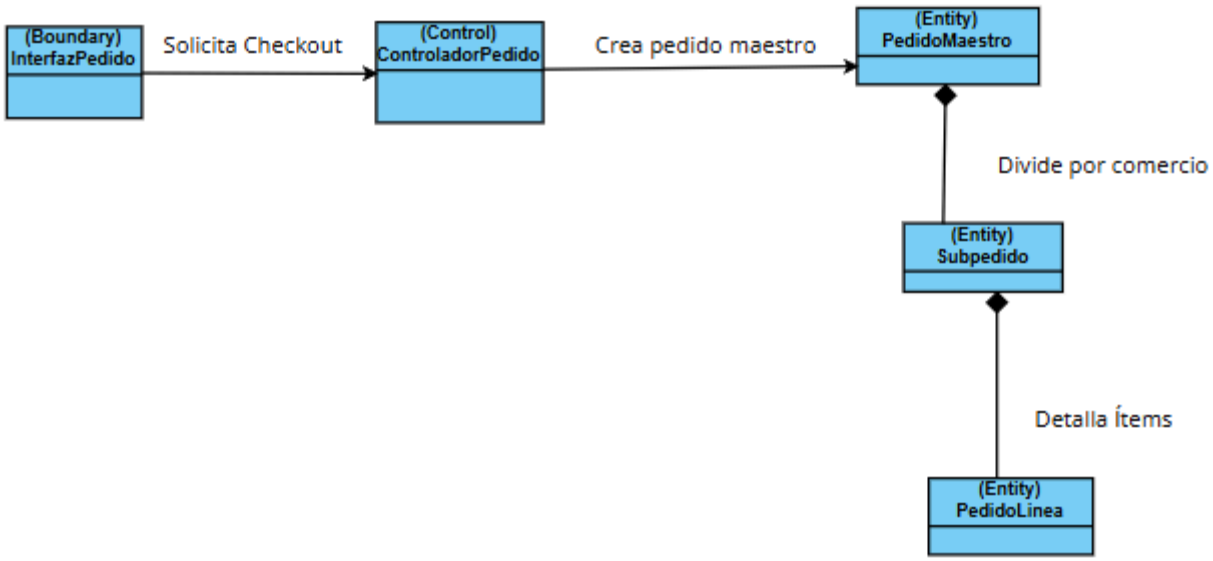
3.2 Realización del caso de uso

Caso de uso (PC2)	CU-004 Confirmar compra y generar pedido(s)
Actor principal	Cliente
Módulos externos involucrados	M6 Pagos para procesar el pago, M4 inventario para verificar y deducir stock y M2 para validar que los comercios estén activos antes de generar los pedido.

Estereotipo	Clase	Responsabilidad en este caso de uso
Límite	InterfazPedido	Manejar la solicitud del cliente en donde se desea finalizar la compra y confirme la transacción.
Control	ControladorPedido	Administrar el proceso de checkout delegando las validaciones correspondientes al inventario, para que se procesen los pagos y transformar un carrito a pedido.
Entidad	PedidoMaestro	Representar el origen o registro original del pedido realizado por el cliente tras el checkout.

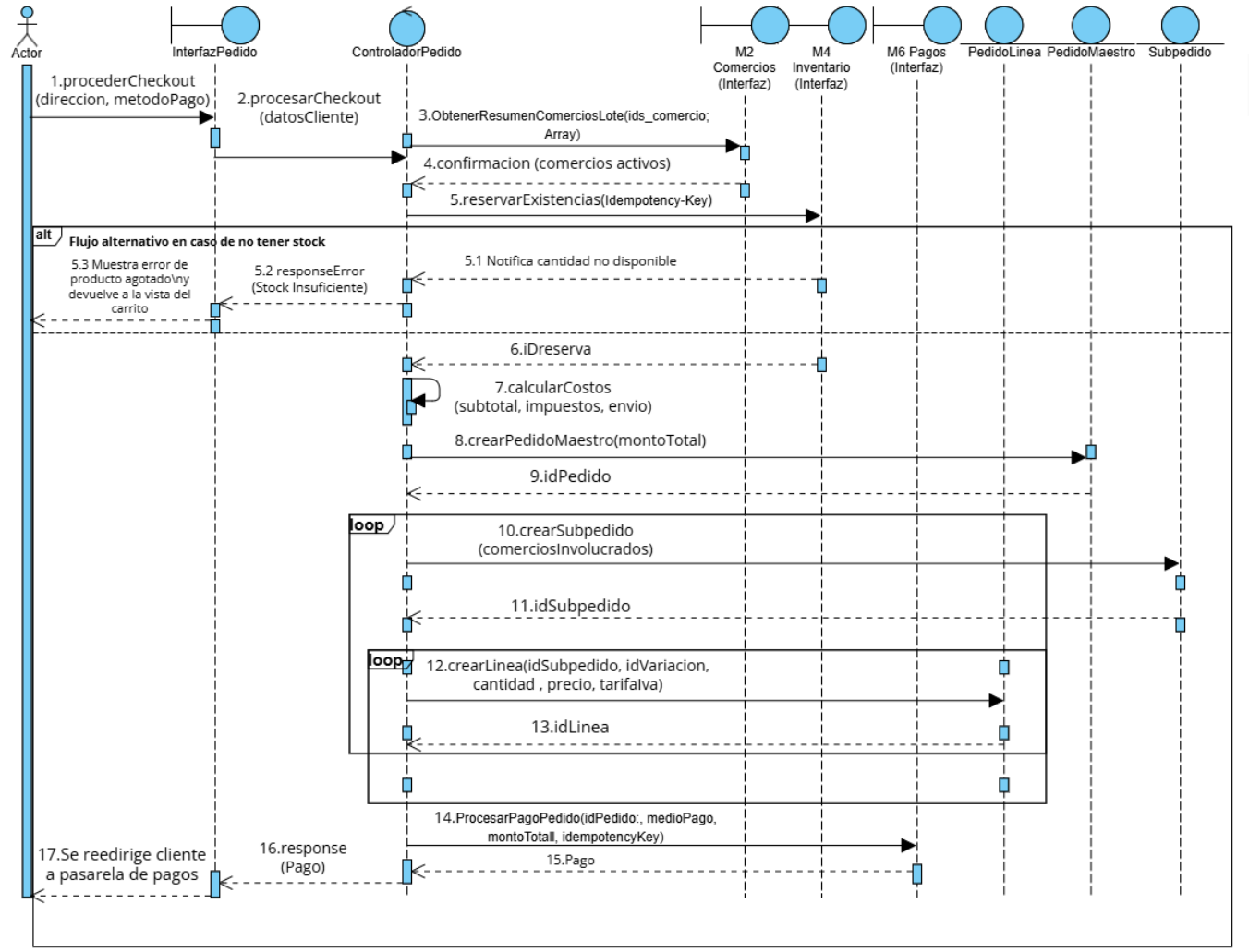
Estereotipo	Clase	Responsabilidad en este caso de uso
Entidad	Subpedido	Representar las divisiones del pedido en caso de ser de diferentes comercios
Entidad	PedidoLinea	Representar el detalle de cada uno de los productos dentro de carrito después del checkout

Figura 3. *Diagrama de caso clases de CU-004 Confirmar compra y generar pedido(s)*



Nota: Elaboración propia en herramienta Visual Paradigm

Figura 4. *Diagrama de caso secuencia de* CU-004 Confirmar compra y generar pedido(s)

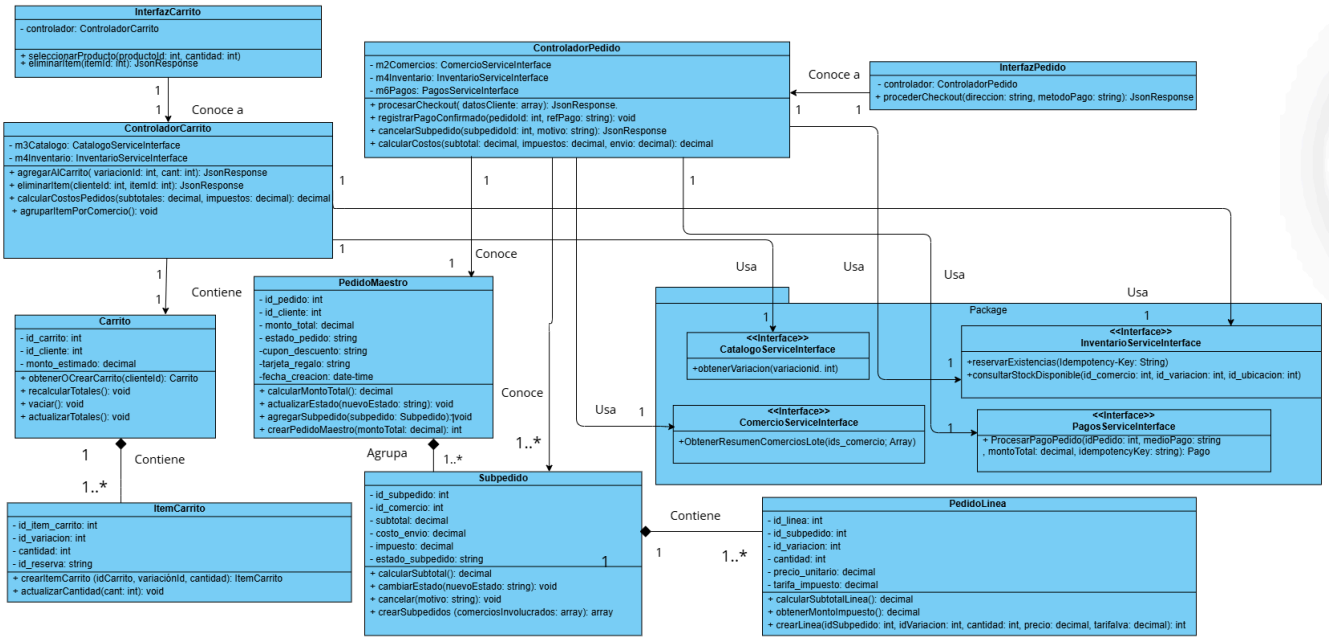


Nota: Elaboración propia en herramienta Visual Paradigm

4. Diagrama de clases de diseño del módulo

Diagrama completo del módulo listo para programar: con tipos, visibilidad, firmas de métodos y navegabilidad.

Figura 5. *Diagrama de clases general del M5*



Nota: Elaboración propia en herramienta Visual Paradigm

4.1 Resumen de clases

Clase	Capa	Atributos principales	Métodos principales (firma)	Relaciones y navegabilidad
InterfazCarrito	Límite	- controlador: ControladorCarrito	+ seleccionarProducto(variacionId : int, cantidad: int): JsonResponse + eliminarItem(itemId: int): JsonResponse	Conoce a ControladorCarrito (1)
ControladorCarrito	Control	- m3Catalogo: CatalogoServiceInterface - m4Inventario: InventarioServiceInterface	+ agregarAlCarrito(variacionId: int, cant: int): JsonResponse + eliminarItem(clienteId: int, itemId: int): JsonResponse + calcularCostosPedidos(subtotales: decimal, impuestos: decimal): decimal + agruparItemPorComercio(): void	Conoce a CatalogoServiceInterface (1), InventarioServiceInterface (1), Carrito (1)
Carrito	Entidad	- id_carrito: int - id_cliente: int -monto_estimado: decimal	+ obtenerOCrearCarrito(clienteId: int): Carrito + recalcularTotales(): void + vaciar(): void	Contiene ItemCarrito (1..*)
ItemCarrito	Entidad	-id_item_carrito: int - id_variacion: int - cantidad: int - id_reserva: string	+ crearItemCarrito(idCarrito: int, variacionId: int, cantidad: int): ItemCarrito + actualizarCantidad(cant: int): void	Pertenece a Carrito (1)

InterfazPedido	Límite	- controlador: ControladorPedido	+ procederCheckout(datosEntrega: array): JsonResponse	Conoce a ControladorPedido (1)
ControladorPedido	Control	- m2Comercios: ComercioServiceInterface - m4Inventario: InventarioServiceInterface - m6Pagos: PagosServiceInterface	+ procesarCheckout(datos: array): JsonResponse + registrarPagoConfirmado(pedidoId: int, refPago: string): void + cancelarSubpedido(subpedidoId: int, motivo: string): JsonResponse + calcularCostos(subtotal: decimal, impuestos: decimal, envio: decimal): decimal	Conoce a interfaces externas (M2, M4, M6), PedidoMaestro (1) y Subpedido (1..*)
PedidoMaestro	Entidad	- id_pedido: int - id_cliente: int - monto_total: decimal -estado_pedido: string -cupon_descuento: string -tarjeta_regalo: string -fecha_creacion: date-time	+ calcularMontoTotal(): decimal + actualizarEstado(nuevoEstado: string): void + agregarSubpedido(subpedido: Subpedido): void + crearPedidoMaestro(montoTotal: decimal): int	Agrupa Subpedido (1..*)
Subpedido	Entidad	- id_subpedido: int	+ calcularSubtotal(): decimal	Pertenece a PedidoMaestro (1), contiene PedidoLinea (1..*)

		- id_pedido: int - id_comercio: int - subtotal: decimal - costo_envio: decimal - impuesto: decimal - estado_subpedido: string	+ cambiarEstado(nuevoEstado: string): void + cancelar(motivo: string): void +crearSubpedido(comerciosInvolucrados: array): array	
PedidoLinea	Entidad	- id_linea: int - id_subpedido: int - id_variacion: int - cantidad: int -precio_unitario: decimal -tarifa_impuesto: decimal	+ calcularSubtotalLinea(): decimal + obtenerMontoImpuesto(): decimal + crearLinea(idSubpedido: int, idVariacion: int, cantidad: int, precio: decimal, tarifalva: decimal): int	Pertenece a Subpedido (1)

4.2 Clases del núcleo y de otros módulos que se usan

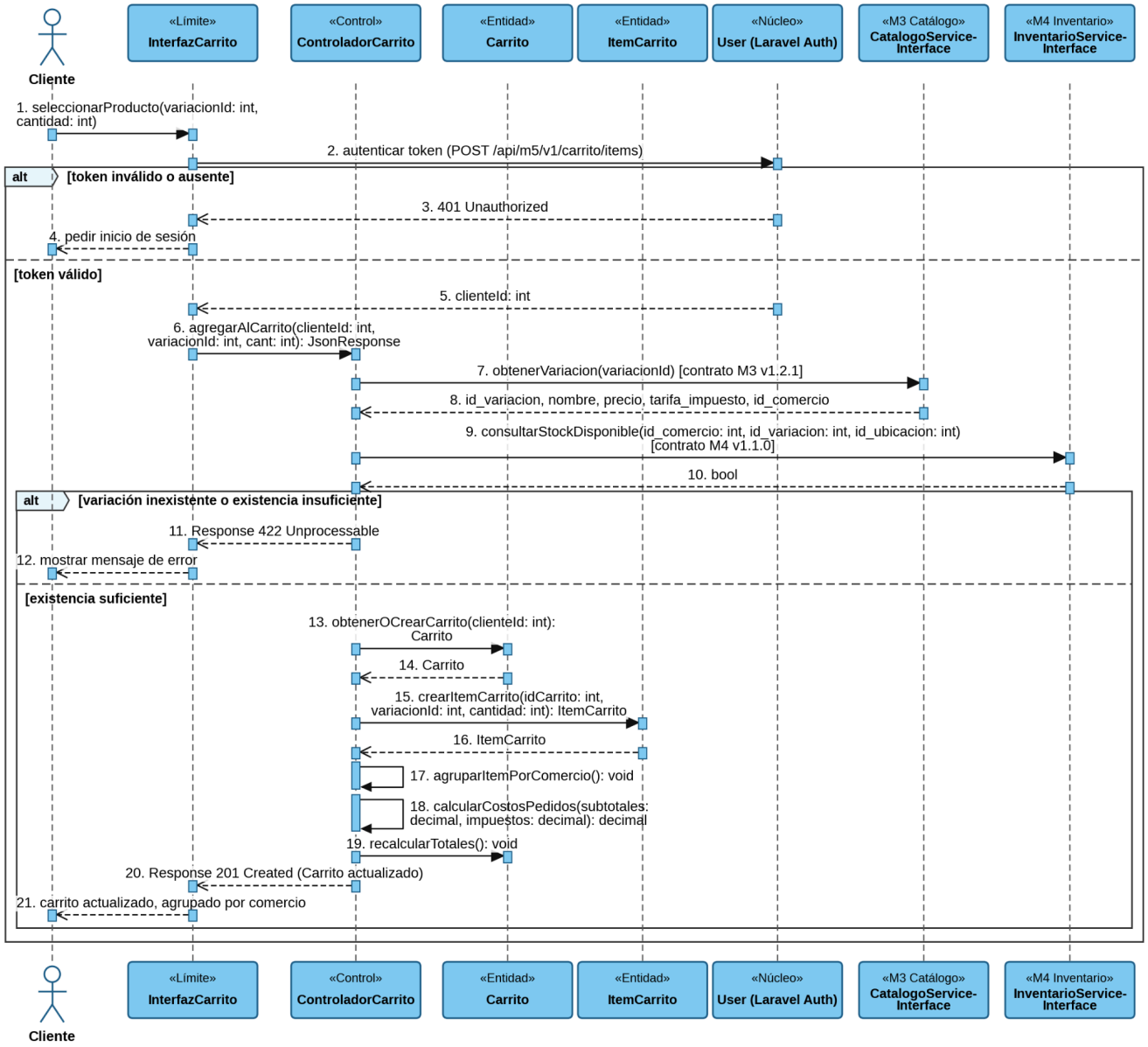
Clase o interfaz externa	Origen	Para qué la usa el módulo
InventarioServiceInterface	M4_Inventario	Se utiliza en dos momentos críticos: en el carrito para ejecutar consultarStockDisponible(id_comercio, id_variacion, id_ubicacion) , y en el checkout para ejecutar reservarExistencias(Idempotency-Key) . Esta última devuelve un idReserva válido por 15 minutos, lo cual resuelve el problema de concurrencia (HALL-02) definido por el equipo.
ComercioServiceInterface	M2_Comercios	Se utiliza de manera sincrónica al momento de procesar un checkout para invocar a ObtenerResumenComerciosLote(ids_comercio) . En busca de crear los subpedidos sólo cuando los comercios encargados confirmen sigan activos en el sistema.
PagosServiceInterface	M6_Pagos	Se utiliza hasta el final del flujo del checkout para invocar + ProcesarPagoPedido(idPedido, medioPago, montoTotal, idempotencyKey). Entrega los datos financieros del PedidoMaestro y retorna la urlPasarelaPago a la que el controlador deberá redirigir al cliente.
CatalogoServiceInterface	M3_Catalogo	De acuerdo a lo que se ha hablado y coordinado entre los subgrupos, el M5 consumirá esta interfaz para tener acceso a la información de los productos, variaciones , precio base y disponibilidad de dichos productos ya que el M5 no es el dueño de esa información como tal.
User	Núcleo	Esta sección corresponde a la forma de autentificación de Laravel(app/Models/User.php). Aquí se representa al actor “Cliente” y provee el clienteId que el módulo recibe en los controladores para asociarlo tanto a la entidad Carrito como al PedidoMaestro.

5. Diagramas de interacción actualizados

Diagramas de secuencia de los casos de uso del PC2 actualizados con las clases de diseño, incluyendo las llamadas a otros módulos.

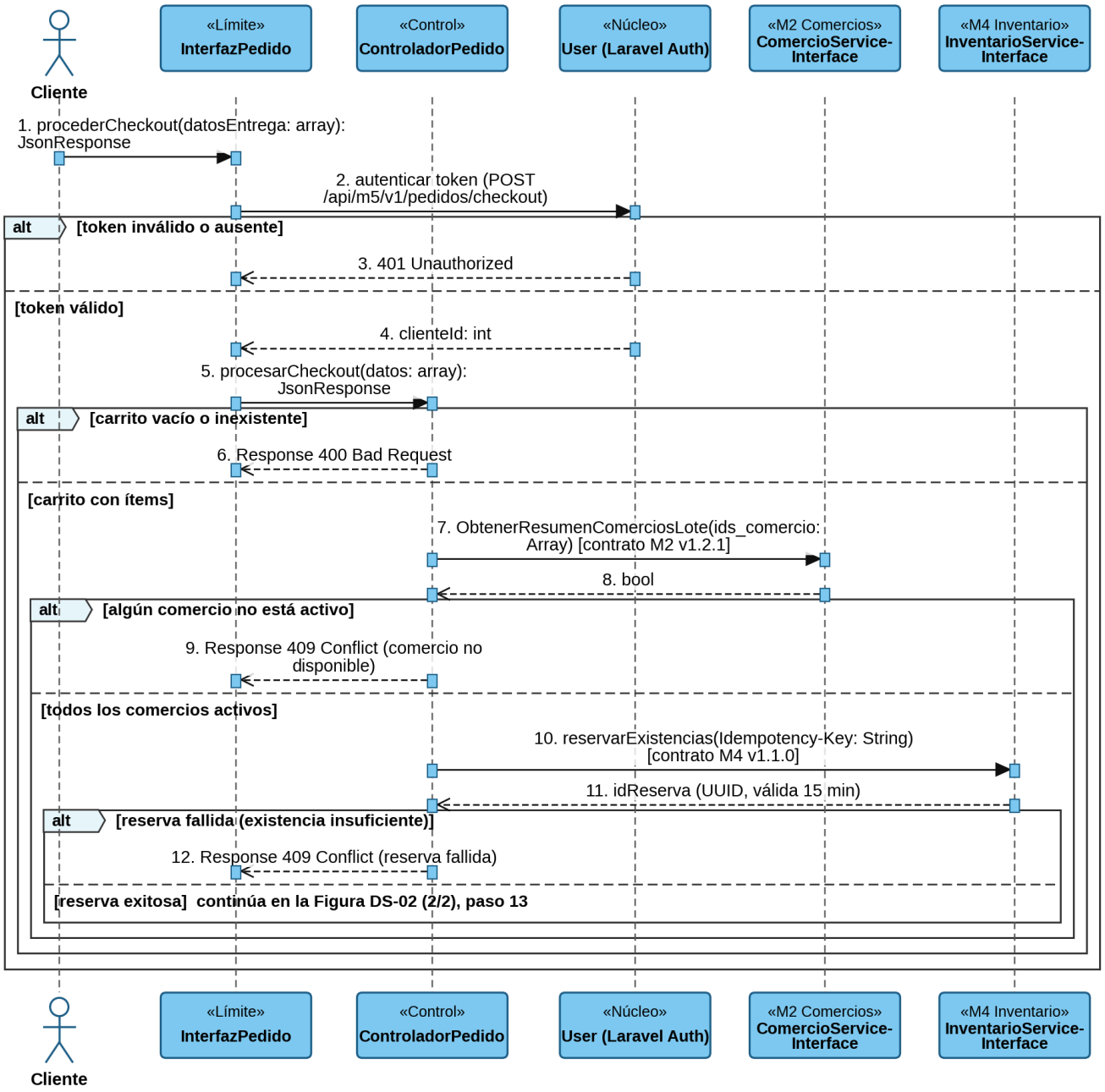
Diagrama	Caso de uso	Módulos externos llamados	Operación o evento del contrato usado
DS-01	CU-001 Gestionar carrito de compras (agregar ítem)	M3, M4	M3 v1.2.1, obtenerVariacion(variacionId) (precio, tarifa_impuesto, id_comercio). M4 v1.1.0, consultarStockDisponible(id_comercio: int, id_variacion: int, id_ubicacion: int).
DS-02	CU-004 Confirmar compra y generar pedido(s)	M2, M4, M6	M2 v1.2.1, ObtenerComercioActivo. M4 v1.1.0, reservarExistencias(Idempotency-Key: String) (reserva de 15 min). M6 v1.0.0, ProcesarPagoPedido(idPedido: int, medioPago: String, montoTotal: decimal, idempotencyKey: String) (devuelve urlPasarelaPago). Evento publicado: PedidoConfirmado v1.0.0.
DS-03	CU-004 Confirmar compra y generar pedido(s) (confirmación de pago)	M6	M6 v1.0.0, ConfirmarPagoRecibido. Evento consumido: PagoConfirmado, atendido en registrarPagoConfirmado(pedidoId: int, refPago: string). Evento publicado: PedidoPagado v1.0.0.

Figura 6. *DS-01 · Diagrama de secuencia de diseño: agregar ítem al carrito (CU-001)*



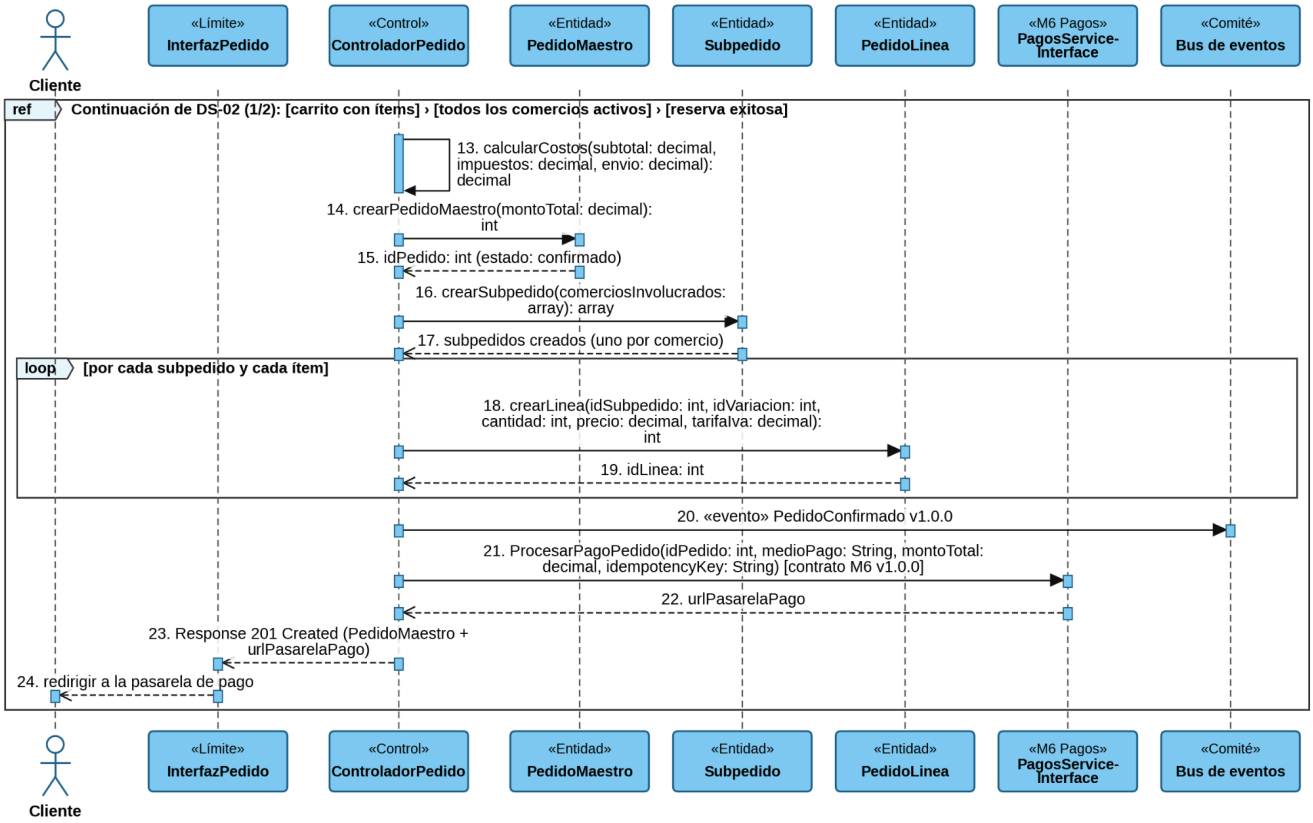
Nota. Elaboración propia mediante la herramienta Visual Paradigm.

Figura 7. DS-02 (1/2) · Diagrama de secuencia de diseño: confirmar compra y generar pedido(s) (CU-004), validaciones



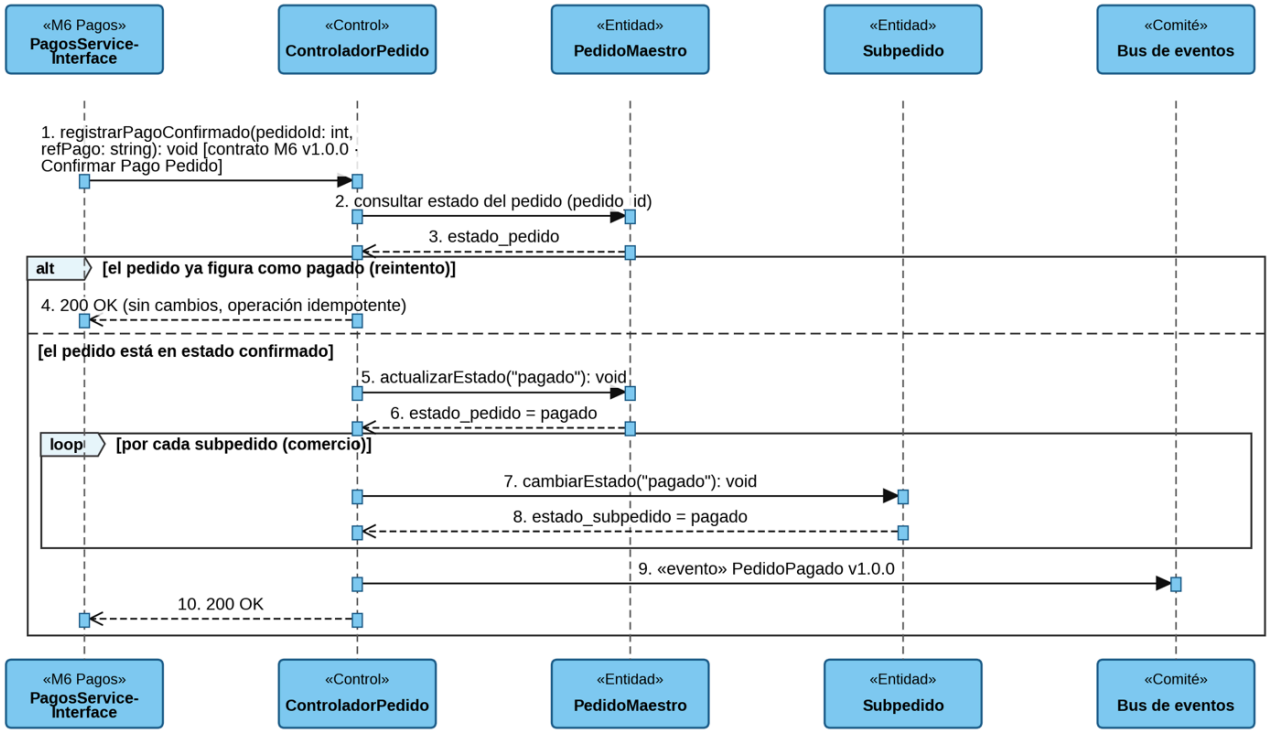
Nota. Elaboración propia mediante la herramienta Visual Paradigm

Figura 8. *DS-02 (2/2) · Diagrama de secuencia de diseño: confirmar compra y generar pedido(s) (CU-004), creación del pedido y cobro.*



Nota. Elaboración propia mediante la herramienta Visual Paradigm

Figura 9. *DS-03 · Diagrama de secuencia de diseño: confirmación de pago recibida de M6 (CU-004)*



Nota. Elaboración propia mediante la herramienta Visual Paradigm

Flujos de error representados:

Diagrama	Condición	Respuesta del módulo	Código del contrato
DS-01	Token ausente o inválido	Se rechaza antes de llegar al controlador.	401 Unauthorized
DS-01	Variación inexistente o existencia insuficiente en M4	No se crea el ítem; se informa al cliente.	422 Unprocessable
DS-02	Token ausente o inválido	Se rechaza antes de llegar al controlador.	401 Unauthorized
DS-02	Carrito vacío o inexistente	No se inicia el checkout.	400 Bad Request
DS-02	Algún comercio del carrito no está activo (M2)	No se crean pedido ni subpedidos.	409 Conflict
DS-02	M4 no puede reservar la existencia	No se crean pedido ni subpedidos.	409 Conflict (reserva fallida)
DS-03	M6 reenvía una confirmación de un pedido ya pagado	Se ignora sin cambios (operación idempotente).	200 OK

6. Modelo de datos del módulo

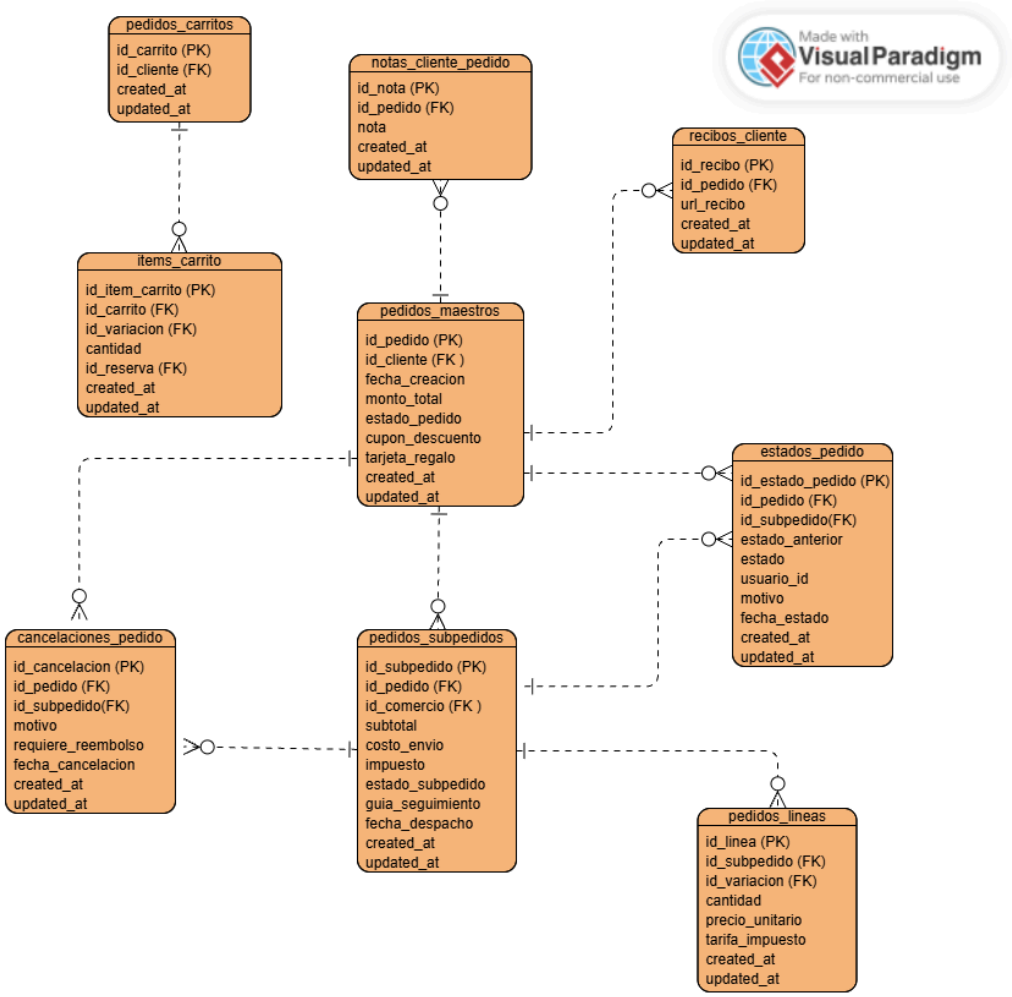
Modelo en tercera forma normal, implementado como migraciones de Laravel, con restricciones de unicidad y verificación que implementan las reglas de negocio. Se entrega además el SQL resultante.

El modelo de Datos del Módulo para M5, ha sido creado como documento aparte con el nombre de Modelo de Datos M5 - Carrito y Pedidos y con extensión .docx. Este mismo puede ser consultado en el siguiente enlace el cual contiene el más actualizado: Modelo de Datos M5 - Carrito y Pedidos

6.1 Tablas del módulo

Tabla	Clave primaria	Claves compartidas que guarda	Descripción
pedidos_maestros	id_pedido	id_cliente	Registro formal generado al confirmar una compra. Consolida el pedido unificado.
pedidos_subpedidos	id_subpedido	id_pedido, id_comercio	Desglose del pedido maestro por cada comercio involucrado.
pedidos_lineas	id_linea	id_subpedido, id_variacion	Desglose de productos (variaciones) incluidos en cada subpedido.
pedidos_carritos	id_carrito	id_cliente	Contenedor virtual y temporal donde la persona compradora reúne selecciones.
items_carrito	id_item_carrito	id_carrito, id_variacion, id_reserva	Productos almacenados temporalmente en el carrito (incluye id_reserva en UUID).
estados_pedido	id_estado_pedido	id_pedido, id_subpedido, usuario_id	Registro y trazabilidad de los estados de un pedido en su ciclo de vida.
notas_cliente_pedido	id_nota	id_pedido	Notas especiales brindadas por la persona compradora.
recibos_cliente	id_recibo	id_pedido	Comprobantes de venta temporales.
cancelaciones_pedido	id_cancelacion	id_pedido, id_subpedido	Registro formal de pedidos cuyo proceso fue interrumpido.

Figura 10. *Diagrama Entidad-Relación de Carrito y Pedidos*



Nota. Elaboración propia mediante la herramienta Visual Paradigm.

6.2 Verificación de tercera forma normal

Tabla	1FN (valores atómicos)	2FN (sin dependencias parciales)	3FN (sin dependencias transitivas)	Observaciones
pedidos_maestros	Cumple	Cumple	Cumple	Los atributos dependen únicamente del id_pedido. Atributos atómicos.
pedidos_subpedidos	Cumple	Cumple	Cumple	Desglose independiente por comercio. Evita repetir costos de envío en cada línea.
pedidos_lineas	Cumple	Cumple	Cumple	Atributos atómicos y específicos de la línea, dependiendo solo de id_linea.

Tabla	1FN (valores atómicos)	2FN (sin dependencias parciales)	3FN (sin dependencias transitivas)	Observaciones
pedidos_carritos	Cumple	Cumple	Cumple	Se separa de los ítems para no repetir el id_cliente en cada artículo.
items_carrito	Cumple	Cumple	Cumple	Depende de la clave primaria. El id_reserva es propio del ítem, garantizando atomicidad.
estados_pedido	Cumple	Cumple	Cumple	Registro inmutable de transiciones. Evita tener múltiples estados en la tabla de pedido.
notas_cliente_pedido	Cumple	Cumple	Cumple	Desligado de la tabla maestra para soportar múltiples notas manteniendo integridad.
recibos_cliente	Cumple	Cumple	Cumple	Guarda URLs únicas de comprobantes asociados al pedido.
cancelaciones_pedido	Cumple	Cumple	Cumple	Detalles de cancelación específicos del evento de anulación.

6.3 Restricciones que implementan reglas de negocio

Regla de negocio (PC2)	Tabla	Tipo de restricción	Definición	Nombre en la migración
RN-M5-01 (Monto total no negativo)	pedidos_maestros	check	monto_total >= 0	monto_total_positivo
RN-M5-01 (Subtotal no negativo)	pedidos_subpedidos	check	subtotal >= 0	subtotal_positivo
RN-M5-01 (Costo de envío no neg.)	pedidos_subpedidos	check	costo_envio >= 0	costo_envio_positivo
RN-M5-02 (Impuesto no negativo)	pedidos_subpedidos	check	impuesto >= 0	impuesto_positivo
RN-M5-02 (Tarifa IVA no negativa)	pedidos_lineas	check	tarifa_impuesto >= 0	tarifa_impuesto_positiva
RN-M5-02 (Precio unitario no neg.)	pedidos_lineas	check	precio_unitario >= 0	precio_unitario_positivo
RF-001 (Cantidad mayor a 0)	pedidos_lineas	check	cantidad > 0	cantidad_positiva

Regla de negocio (PC2)	Tabla	Tipo de restricción	Definición	Nombre en la migración
RF-001 (Un carrito por cliente)	pedidos_carritos	unique	Unicidad en id_cliente	pedidos_carritos_id_cliente_unique
RF-001 (Variación única en carrito)	items_carrito	unique	id_carrito, id_variacion	items_carrito_unique
RF-001 (Cantidad en carrito > 0)	items_carrito	check	cantidad > 0	cantidad_positiva_carrito

6.4 Migraciones

Las migraciones correspondientes se encuentran almacenadas en el siguiente directorio: database/migrations/M5_Pedidos

Migraciones a encontrar dentro de esa carpeta:

Archivo de migración	Tabla	Descripción
2026_10_09_013223_create_pedidos_maestros_table.php	pedidos_maestros	Crea la tabla principal de pedidos.
2026_10_09_013224_create_pedidos_subpedidos_table.php	pedidos_subpedidos	Desglose por comercio con campos de envío.
2026_10_09_013225_create_pedidos_lineas_table.php	pedidos_lineas	Crea el detalle de cada artículo.
2026_10_09_013226_create_pedidos_carritos_table.php	pedidos_carritos	Crea el registro del carrito de compras.
2026_10_09_013227_create_items_carrito_table.php	items_carrito	Crea los ítems temporales vinculados al carrito.
2026_10_09_013228_create_estados_pedido_table.php	estados_pedido	Historial de transiciones según req. de auditabilidad 3.6.
2026_10_09_013229_create_notas_cliente_pedido_table.php	notas_cliente_pedido	Almacena las observaciones del comprador.
2026_10_09_013230_create_recibos_cliente_table.php	recibos_cliente	Crea la tabla de enlaces a comprobantes temporales.
2026_10_09_013231_create_cancelaciones_pedido_table.php	cancelaciones_pedido	Registra motivos de anulación (soporta cancelación parcial).

6.5 SQL resultante

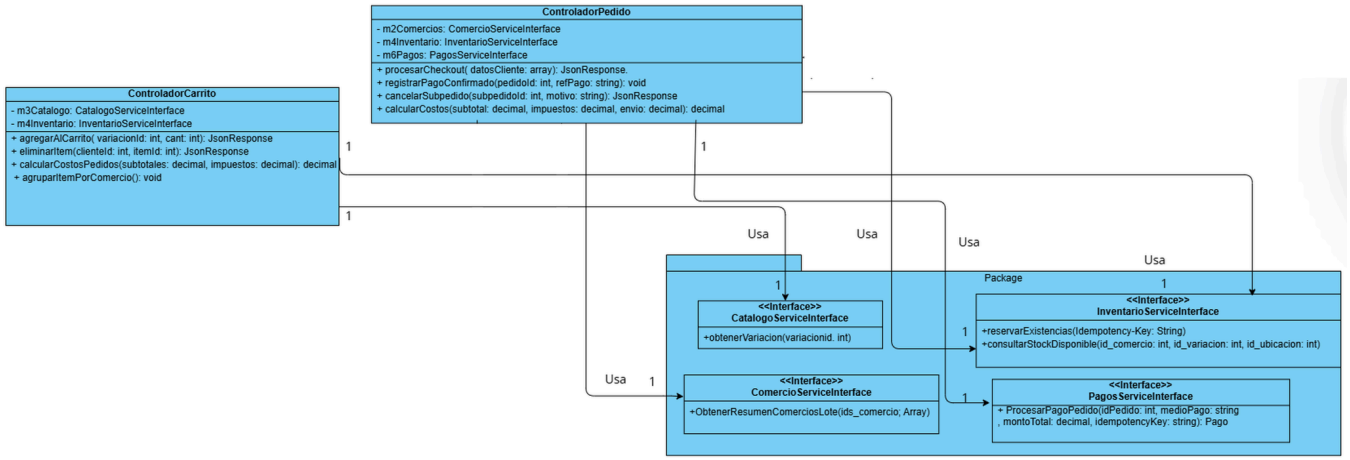
El SQL generado a partir de las migraciones de Laravel ha sido exportado exitosamente usando el comando de simulación. La ruta del archivo en el repositorio es: **db/M5_Pedidos/sql_generated.txt**

7. Patrones de diseño aplicados

7.1 Adapter

Categoría	Estructural
Problema del módulo que resuelve	El módulo M5 necesita consultar precios en M3, verificar estados de tiendas en M2, reservar stock en M4 y solicitar cobros en M6. Para comunicarse con estos módulos, no puede consultar directamente sus tablas ni importar sus modelos Eloquent. Además, debe permitir usar mocks o fakes para hacer pruebas sin tener que modificar el código de M5.
Dónde se aplica	En el paquete de interfaces del diagrama de la Sección 4: CatalogoServiceInterface (M3), ComercioServiceInterface (M2), InventarioServiceInterface (M4) y PagosServiceInterface (M6), las cuales son consumidas mediante las relaciones «Usa» por ControladorCarrito y ControladorPedido.
Justificación	Separa completamente la lógica de M5 de la implementación interna de los demás subgrupos, siguiendo la comunicación por contrato y el ADR-004. El contenedor de inversión de control de Laravel debe poder inyectar el cliente real o el doble de prueba de la sección 9 sin necesidad de cambiar el código de M5.
Alternativa descartada	Llamar directamente a clientes HTTP ni usar modelos de otros módulos dentro de ControladorPedido y ControladorCarrito. Esto haría que el módulo dependa de rutas externas fijas, incumpla las reglas del Comité y dificulte hacer pruebas por separado cuando otro subgrupo todavía no haya desplegado su módulo.

Figura 11. *Controladores del módulo y el paquete de interfaces de contrato donde se aplica el patrón Adapter*

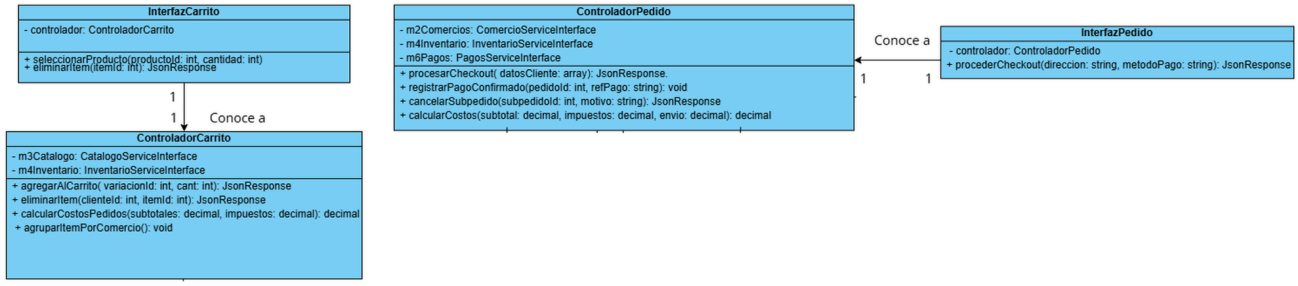


Nota. Fragmento del diagrama de clases de diseño de la sección 4. Elaboración propia mediante la herramienta Visual Paradigm

7.2 Facade

Categoría	Estructural
Problema del módulo que resuelve	Confirmar una compra (checkout) en una plataforma multicomercio requiere varios pasos: verificar que los comercios estén activos en M2, reservar el stock de forma atómica en M4 (ADR-002), calcular los costos, crear el pedido principal, dividirlo en subpedidos por comercio, registrar los productos e iniciar el cobro en M6 (ADR-005). Si InterfazPedido tuviera que realizar todas estas tareas por separado, tendría demasiada lógica de negocio.
Dónde se aplica	En las clases ControladorPedido (a través de su método procesarCheckout) y ControladorCarrito (a través de agregarAlCarrito), que actúan como fachadas frente a las clases límite InterfazPedido e InterfazCarrito.
Justificación	Ofrece un único punto de entrada para la interfaz de usuario en React (ADR-001). La pantalla limite solo debe llamar a procesarCheckout(datosCliente), mientras que la fachada (ControladorPedido) se encarga de coordinar las 4 interfaces externas y las 3 entidades de datos. Todo el proceso debe completarse en menos de 3 segundos (ESC-06).
Alternativa descartada	Hacer que el cliente frontend (React) o la clase límite (InterfazPedido) llame directamente a M2 para validar comercios, luego a M4 para reservar, luego guarde el pedido y finalmente llame a M6. Esto expondría la lógica del proceso en el frontend, aumentaría las llamadas de red y podría dejar el sistema en un estado inconsistente si alguna llamada falla a mitad del proceso.

Figura 12. *Clases límite y controladores del módulo donde se aplica el patrón Facade*

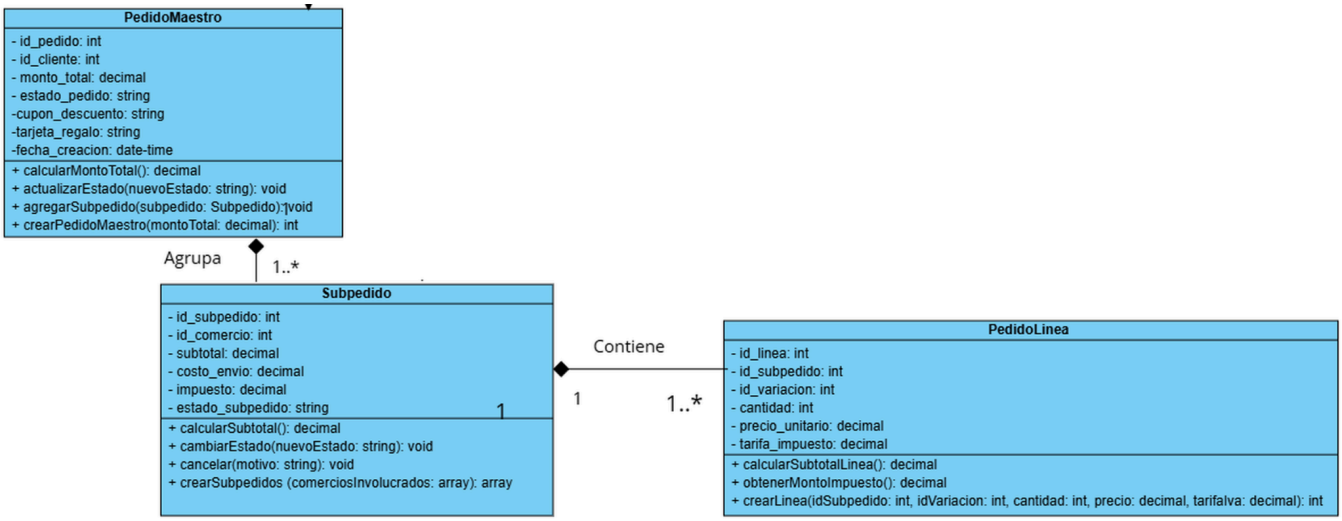


Nota. Fragmento del diagrama de clases de diseño de la sección 4. Elaboración propia mediante la herramienta Visual Paradigm

7.3 Composite (Composición Jerárquica de Pedidos)

Categoría	Estructural
Problema del módulo que resuelve	Según la decisión oficial 3.1 del Comité, una sola compra del cliente debe manejarse como un pedido global (PedidoMaestro) para realizar el cobro, pero debe dividirse en subpedidos por tienda (SubPedido) para gestionar por separado el despacho, el envío y la facturación. Cada subpedido contiene sus artículos (PedidoLinea). Esta estructura permite calcular los subtotales, impuestos y montos totales de cada tienda sin mezclar sus costos.
Dónde se aplica	En las entidades de dominio PedidoMaestro, Subpedido y PedidoLinea del diagrama de la Sección 4, unidas mediante relaciones de composición fuerte (1 a 1..*) y sus métodos de cálculo en cascada: PedidoLinea.calcularSubtotalLinea(), SubPedido.calcularSubtotal() y PedidoMaestro.calcularMontoTotal().
Justificación	Representa la estructura de árbol del pedido multicomercio. Cada nivel se encarga de sus propios cálculos: la línea calcula el precio por cantidad y el IVA (obtenerMontoImpuesto), el subpedido suma el costo de sus líneas y el flete del comercio (calcularSubtotal), y el pedido maestro suma todos los subpedidos y aplica los descuentos globales (calcularMontoTotal).
Alternativa descartada	Usar una sola tabla y clase Pedido con todos los productos mezclados, sin un nivel intermedio de SubPedido. Esto impediría que cada comercio pueda despachar, facturar o cancelar su parte de forma independiente, sin afectar a los demás comercios de la misma compra. Además, incumpliría la decisión 3.1 del PC2.

Figura 13. *Jerarquía PedidoMaestro, Subpedido y PedidoLinea donde se aplica el patrón Composite*



Nota. Fragmento del diagrama de clases de diseño de la sección 4. Elaboración propia mediante la herramienta Visual Paradigm.

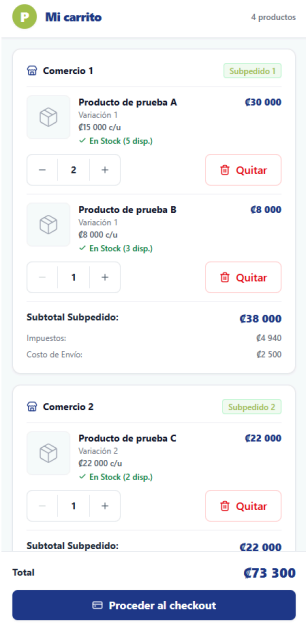
8. Prototipo de interfaz del módulo, validado

8.1 Pantallas del prototipo

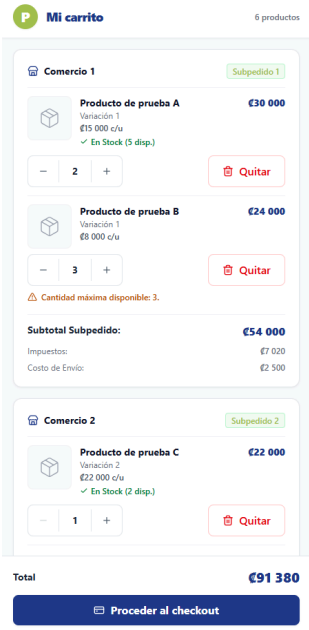
Pantalla	Caso de uso	Actor	Ubicación en el repositorio
Carrito: productos agrupados por comercio, cambio de cantidad, quitar ítem y subtotales	CU-001	Cliente	frontend/src/modules/M5_Pedidos/views/CarritoView.jsx
Checkout: dirección, método de pago, cupón, tarjeta de regalo, nota y resumen por comercio	CU-004	Cliente	frontend/src/modules/M5_Pedidos/views/CheckoutView.jsx
Pedido generado: estado «Pendiente de Pago» y redirección a la pasarela	CU-004	Cliente	frontend/src/modules/M5_Pedidos/views/PedidoPendienteView.jsx
Alternativa: producto agotado, aviso en el carrito	CU-004	Cliente	frontend/src/modules/M5_Pedidos/M5App.jsx (simular=stock)
Alternativa: comercio suspendido, productos deshabilitados	CU-004	Cliente	frontend/src/modules/M5_Pedidos/M5App.jsx (simular=comercio)

El prototipo usa React y Tailwind CSS, el stack acordado por el Comité. Se comprobó en un navegador a 360 píxeles de ancho que ninguna de las cinco vistas tiene desplazamiento horizontal. Los productos, comercios, precios y costos de envío son datos de ejemplo. Las alternativas de los pasos 4 y 5 del CU-004 se activan agregando simular=stock o simular=comercio a la dirección de la página. No se prototipó la selección del producto desde el catálogo, la aplicación del cupón (caso de uso extendido del CU-001) ni la cancelación y devolución de pedidos.

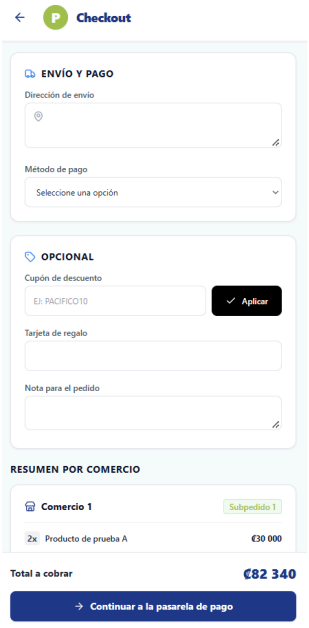
Capturas del prototipo (360 px de ancho, datos de ejemplo; no son resultados de la validación)



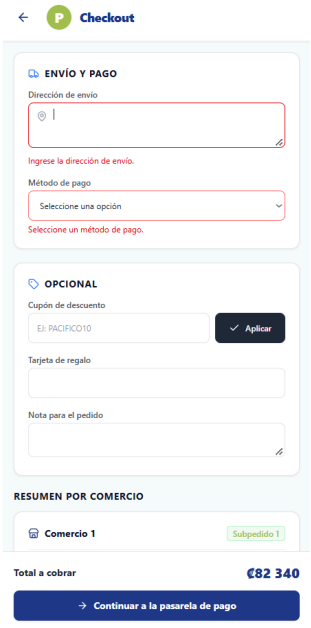
Carrito



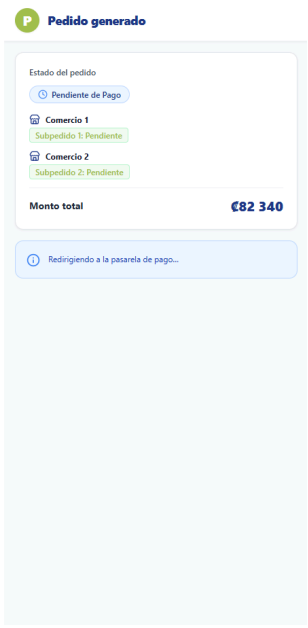
Cantidad máxima disponible



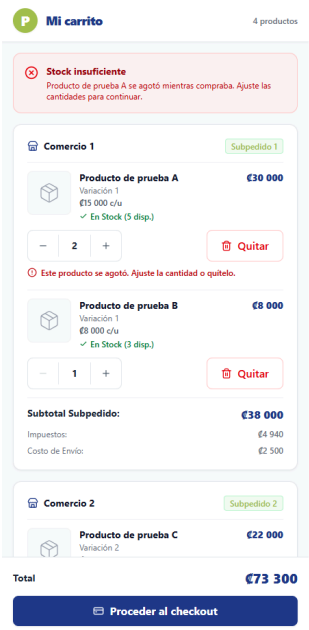
Checkout



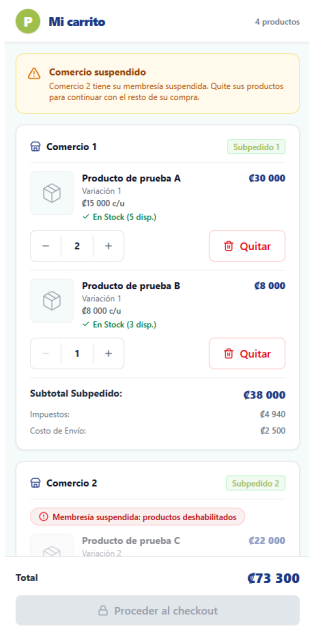
Validación de campos



Pedido pendiente de pago



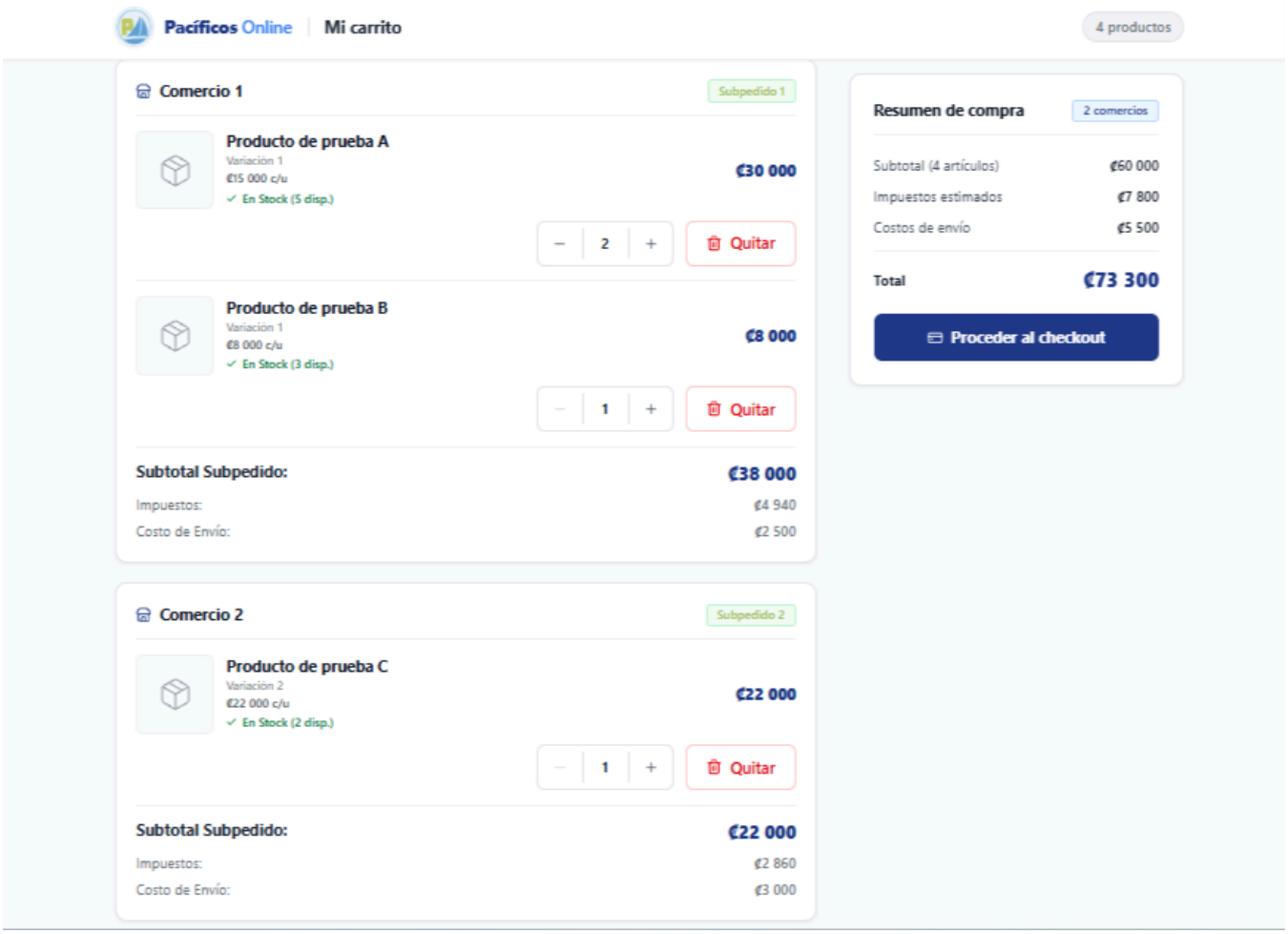
Alternativa: sin stock



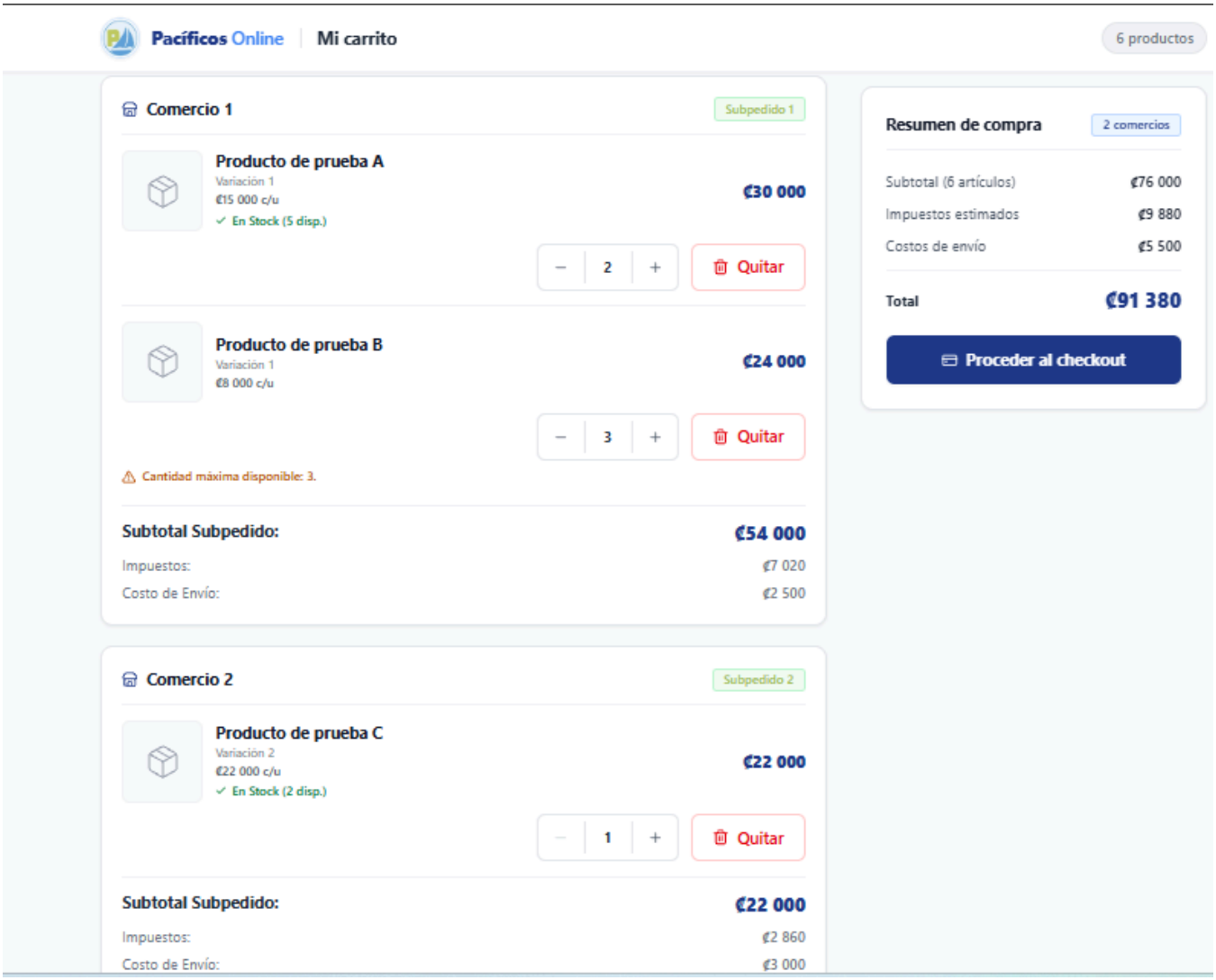
Alternativa: comercio suspendido

Capturas del Prototipo en versión web:

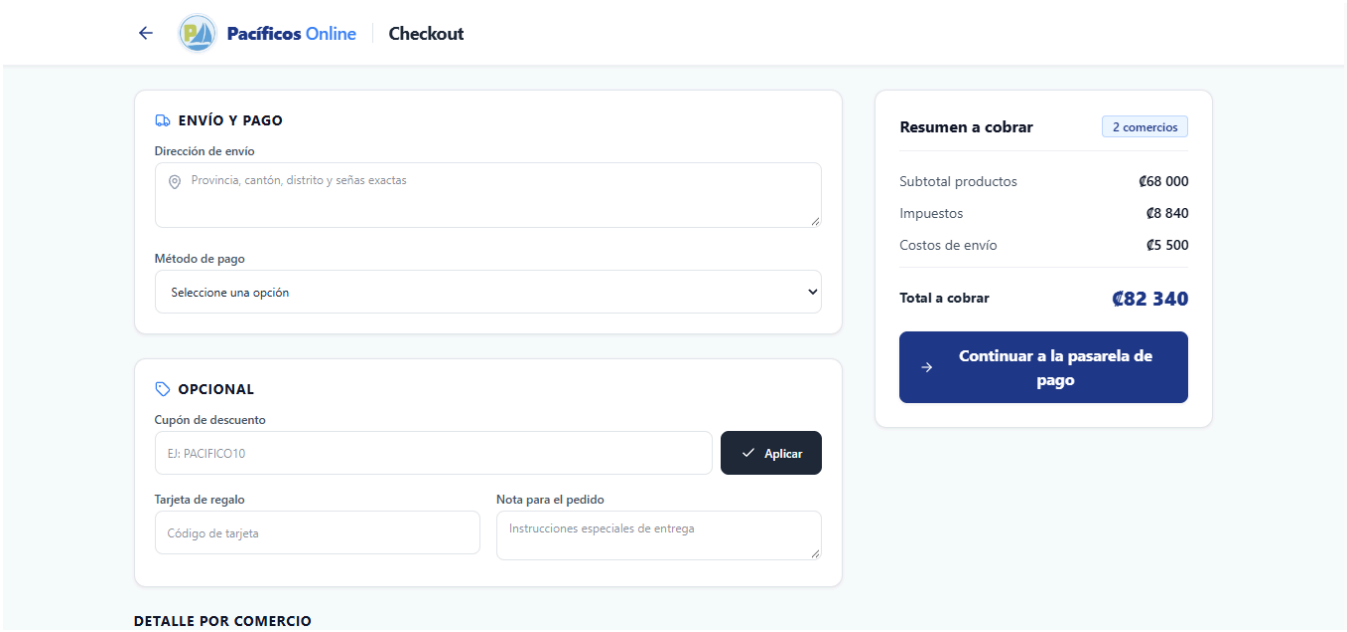
Carrito



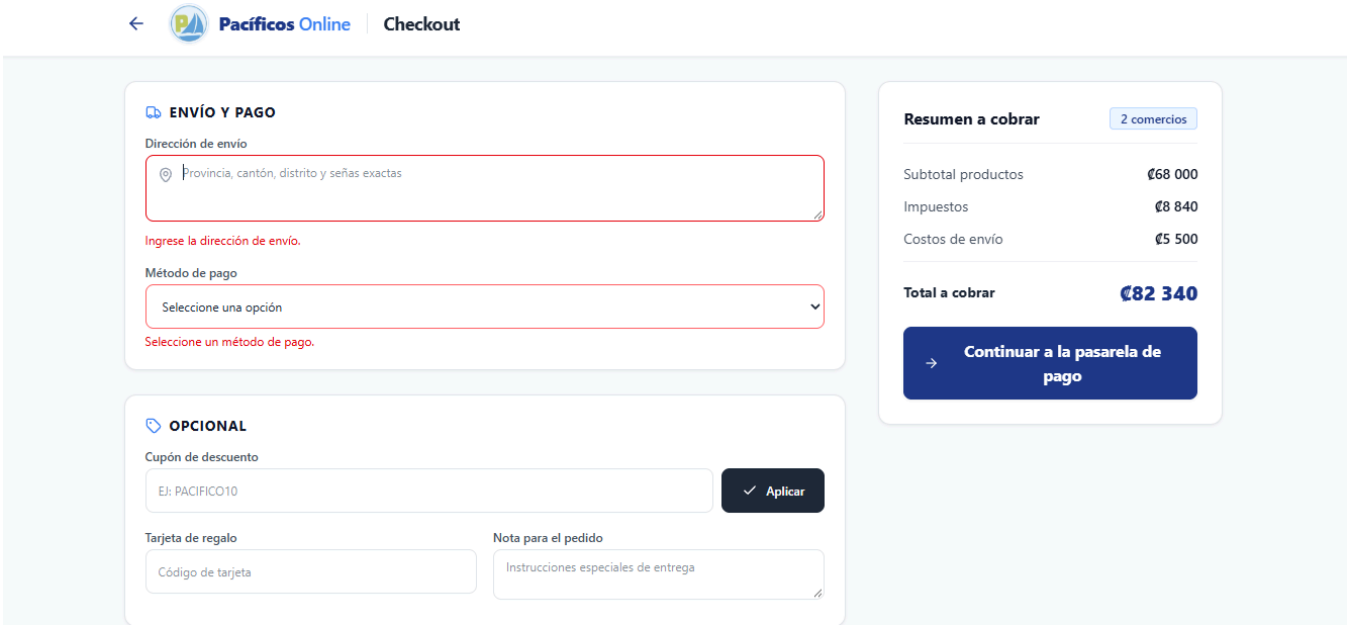
Cantidad máxima disponible



Checkout



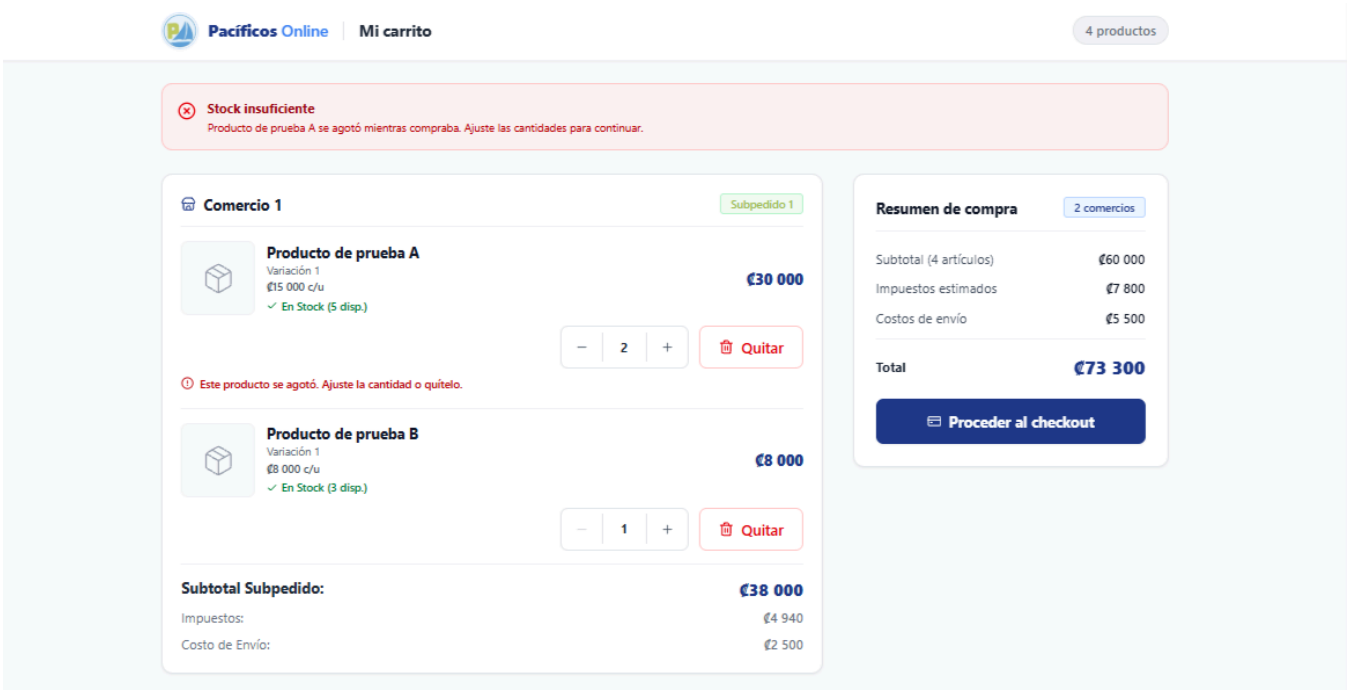
Validación de Campos



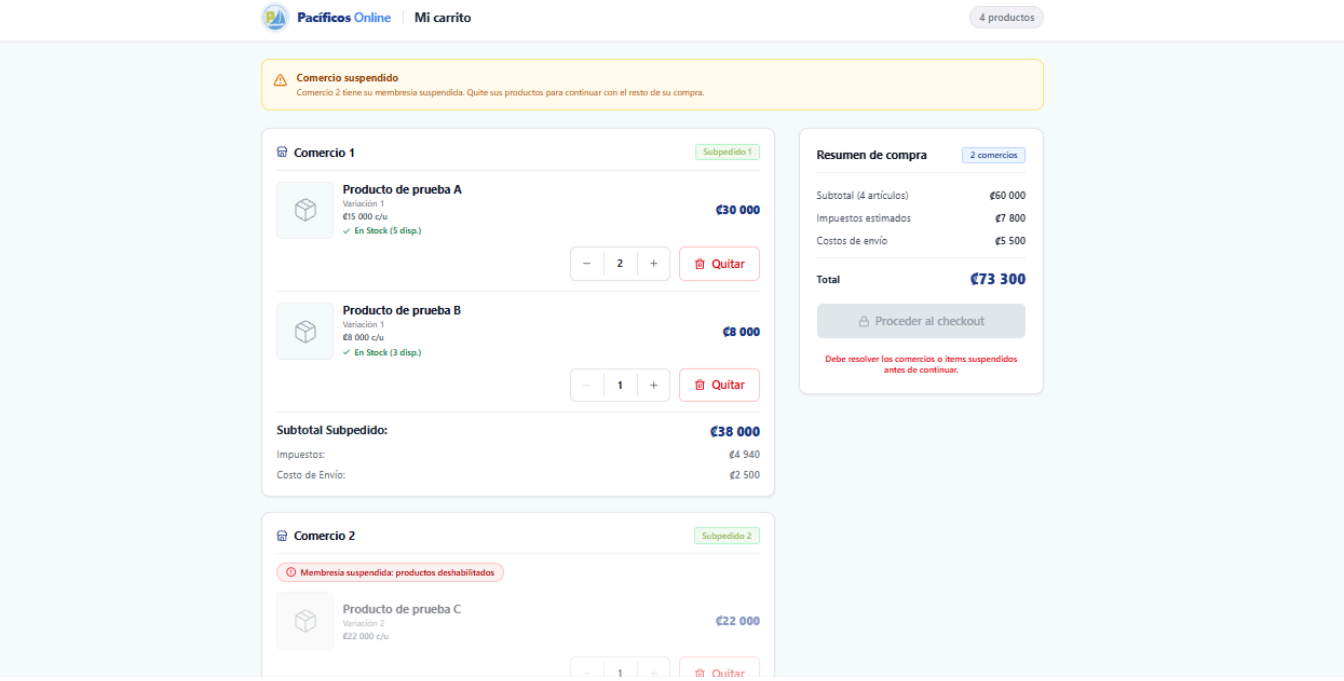
Pedido pendiente de pago



Alternativa: Sin Stock



Alternativa: comercio suspendido



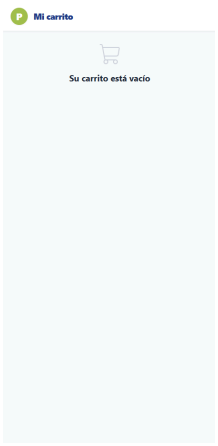
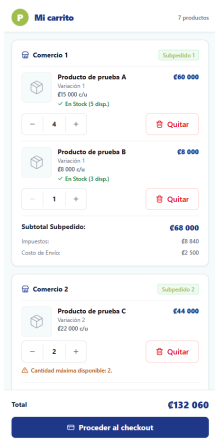
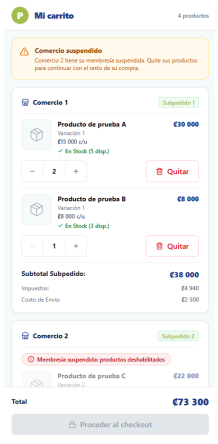
8.2 Validación

Fecha	11 de Octubre 2026 8:35 p.m
Participantes	Participante 1 y Participante 2 (rol de Cliente)
Método	Prueba de tareas con recorrido guiado: cada participante, en el rol de Cliente, usa el prototipo a 360 px de ancho; la persona evaluadora registra si completa cada tarea, dónde duda y qué comenta.
Tareas evaluadas	1. Cambiar la cantidad de un producto, quitar otro y ver los subtotales por comercio. 2. Intentar superar la cantidad máxima disponible. 3. Ir al checkout sin llenar los campos y corregir los errores señalados. 4. Completar el checkout e identificar el estado del pedido y el paso siguiente. 5. Con simular=stock: explicar qué producto se agotó y qué haría. 6. Con simular=comercio: explicar por qué no puede continuar y qué haría.

Resultados por tarea

Tarea	Participante 1	Participante 2
1	Lo completó. Su subtotal fue de ₡47310.	Lo completó. Su subtotal fue de ₡72170.
2	En el producto C quiso poner 3, pero solo hay 2.	En el producto A puso 4 unidades.
3	Eliminó todos los artículos del carrito; el carrito quedó vacío.	Aparecieron los mensajes «Ingrese la dirección de envío» y «Seleccione un método de pago».
4	Dejó los artículos como estaban, fue al checkout, escribió «Mi casa» en Dirección, llenó varios campos, continuó a la pasarela de pago y logró la compra.	Pudo generar el pedido sin problemas.

5	No le permitió comprar.	No le permitió comprar.
6	No le permitió comprar del comercio 2; quitó el producto del comercio 2, lo volvió a intentar y generó el pedido.	No le permitió comprar del comercio 2; quitó el producto del comercio 2, lo volvió a intentar y generó el pedido.

Hallazgo	Pantalla	Cambio realizado	Estado
En la tarea 3, el participante 1 eliminó todos los artículos y dejó el carrito vacío; la tarea pedía ir al checkout y continuar sin llenar los campos. La pantalla de carrito vacío solo muestra el mensaje «Su carrito está vacío» y no ofrece ninguna acción.		Se agregó un botón de acción "Volver a la tienda o Explorar catálogo" debajo del mensaje de carrito vacío para que el cliente no quede en un callejón sin salida.	Aplicado
En la tarea 2, el participante 1 intentó poner 3 unidades del producto C, que solo admite 2, y el participante 2 subió el producto A a 4 unidades, sin llegar a su máximo de 5. El aviso de cantidad máxima aparece únicamente al llegar al tope.		Se mantiene el aviso al llegar al tope, destacando el borde e inhabilitando el botón (+) al alcanzar el límite.	Validado
En la tarea 6, los dos participantes no pudieron comprar del comercio 2, quitaron su producto, reintentaron y generaron el pedido.		Ninguno.	Validado

9. Dobles de prueba de las dependencias externas

Permiten avanzar sin esperar a los otros subgrupos: sustituyen a cada módulo o servicio externo del que depende el módulo.

Dependencia	Contrato y versión que imita	Tipo de doble	Escenarios que cubre	Ubicación en el repositorio	Se reemplaza cuando
M2 — ObtenerResumenComerciosLote	M2 — v1.0.0	Mock de Interfaz	**Éxito:** Retorna estado activo. **Error 409:** Comercio suspendido o inactivo.	tests/Feature/M5_Pedidos/M5DependenciesMockTest.php	El contrato de Comercios esté desplegado en integración.
M3 — obtenerVariacion	M3 — v1.0.0	Mock de Interfaz	**Éxito:** Retorna id_variacion, nombre, precio, tarifa_impuesto, id_comercio. **Error 422:** Variación inexistente.	tests/Feature/M5_Pedidos/M5DependenciesMockTest.php	El contrato de Catálogo esté desplegado en integración.
M4 — ConsultarStockDisponible	M4 — v1.0.0	Mock de Interfaz	**Éxito:** Retorna disponibilidad (true). **Error 422:** Existencia insuficiente.	tests/Feature/M5_Pedidos/M5DependenciesMockTest.php	El contrato de Inventario esté desplegado en integración.
M4 — reservarExistencias	M4 — v1.0.0	Mock de Interfaz	**Éxito:** Retorna id_reserva (UUID) y expiracion (TTL 15 min). **Error 409:** Falla de concurrencia/Reserva fallida.	tests/Feature/M5_Pedidos/M5DependenciesMockTest.php	El contrato de Inventario esté desplegado en integración.
M6 —ProcesarPagoPedido	M6 — v1.0.0	Mock de Interfaz	**Éxito:** Retorna urlPasarelaPago y transaccion_id. **Error:** Pago rechazado por la pasarela.	tests/Feature/M5_Pedidos/M5DependenciesMockTest.php	El contrato de Pagos esté desplegado en integración.

Nota. Los dobles se implementan inyectando las interfaces PHP (ej. M4_Inventario_Interfaz) en el contenedor de servicios de Laravel mediante $this->app->instance(...). Se conmuta a la implementación real registrando el binding de la clase concreta en el AppServiceProvider.